SQL Business Intelligence Report

Airport Analytics Database • Data Analysis Portfolio

Prepared by: Abhishek Chauhan

Executive Summary

This report presents all 8 SQL queries developed against the Airport Analytics database to address key business challenges across Flight Operations, Airline Revenue, Passenger CRM, and Route Strategy. Each query translates raw transactional flight data into actionable business insights.

Database: Airport_data **Total Queries:** 8 **Tables:** Airports, Airlines, Flights, Passengers, Tickets

Query Summary Overview

All 8 business problems solved, the SQL technique used, and the business impact category.

#	Query Name	Section	SQL Technique	Impact
1	Busiest Airport by Take offs	Operations	`JOIN + GROUP BY + FETCH NEXT`	Capacity
2	IndiGo Flights with Airport Names	Operations	`Self-join on Airports (ao & ad)`	Ops
3	Flight Duration & Category	Operations	`CTE + DATEDIFF + CASE WHEN`	Ops
4	Total Tickets Sold per Airline	Revenue	`3-table JOIN + COUNT + GROUP BY`	Revenue
5	Top Airline per Departing Airport	Strategy	`RANK() OVER PARTITION BY`	Strategy
6	Highest Ticket Price per Route	Revenue	`CTE + Correlated Subquery`	Revenue
7	Passenger First & Last Flight Dates	CRM	`MIN / MAX / COUNT + GROUP BY`	CRM
8	Top Spender per Frequent Flyer Tier	CRM	`RANK() OVER PARTITION BY + SUM`	CRM

Section 1 — Flight Operations

These queries give operations teams visibility into airport capacity, IndiGo's route network, and per-flight duration categorisation — driving scheduling, staffing, and service design decisions.

Query 1 • Busiest Airport by Number of take offs

Business Problem	Operations management needs to identify which airport handles the most departing flights to allocate ground staff, gates, and turnaround resources most effectively.

Business Value	Enables airport authorities and airlines to prioritise infrastructure investment, gate management, and peak-hour staffing at the highest-traffic departure hub.
Key Insight	**FETCH NEXT 1 ROW ONLY (SQL Server) paired with ORDER BY extracts the single busiest airport in one clean pass — no subquery or window function required.**

SQL Query

```
SELECT a.Name,
    COUNT(*) AS [total takeoff flights]
FROM Airports a
JOIN Flights f
    ON a.AirportID = f.Origin
GROUP BY a.Name
ORDER BY [total takeoff flights] DESC
OFFSET 0 ROWS
FETCH NEXT 1 ROW ONLY;
```

Query 2 • IndiGo Flights with Full Origin & Destination Airport Names

Business Problem	The scheduling team needs all IndiGo-operated flights displayed with readable airport names instead of raw numeric IDs — for route reviews, presentations, and marketing material.
Business Value	Provides a clear network map of IndiGo operations. Essential for slot negotiation, timetable publishing, and partnership discussions with other carriers.
Key Insight	**The Airports table is joined TWICE using different aliases (ao for origin, ad for destination) — a self-join pattern that resolves both names in a single query without any subquery.**

SQL Query

```
SELECT a.Name   AS Airline,
    ao.Name  AS Origin_Airport,
    ad.Name  AS Destination_Airport
FROM Airlines a
JOIN Flights f   ON a.AirlineID  = f.AirlineID
JOIN Airports ao ON ao.AirportID = f.Origin
JOIN Airports ad ON ad.AirportID = f.Destination
WHERE a.Name = 'IndiGo';
```

Query 3 • Flight Duration & Category (Short / Medium / Long)

Business Problem	The product and pricing team needs every flight categorised by total duration to design appropriate in-flight service levels, meal policies, and fare tier structures.
Business Value	Short-haul flights get snack packs; long-haul get full meal service. Duration buckets directly drive catering, crew rostering, and dynamic pricing model inputs.
Key Insight	**CTE pre-computes DATEDIFF hours first, keeping logic clean. Outer CASE WHEN then assigns labels — separating computation from categorisation makes the query easy to audit and extend.**

SQL Query

```
WITH hours_ctry AS (
    SELECT f.FlightID          AS flight_id,
           a.Name              AS Airline_name,
           DATEDIFF(hour, f.DepartureTime, f.ArrivalTime) AS total_time
    FROM Airlines a
    JOIN Flights f ON a.AirlineID = f.AirlineID
)
```

```
SELECT flight_id,
       Airline_name,
       CASE
           WHEN total_time < 2              THEN 'Short'
           WHEN total_time BETWEEN 2 AND 5 THEN 'Medium'
           ELSE                                 'Long'
       END AS time_duration
FROM hours_ctry;
```

Section 2 — Airline Revenue & Route Strategy

Revenue and strategic queries that surface ticket volumes per airline, the dominant carrier at each airport, and the maximum price achieved on every route — essential inputs for pricing, partnerships, and network expansion.

Query 4 • Total Tickets Sold per Airline

Business Problem	Revenue leadership needs to know how many tickets each airline has sold to benchmark performance, set commission budgets, and negotiate preferential gate or slot arrangements.
Business Value	A clear revenue leaderboard across all carriers. High-volume airlines may qualify for co-marketing agreements, preferential gate assignments, or lounge placement deals.
Key Insight	**Requires a 3-table JOIN chain (Airlines → Flights → Tickets) because no direct link exists between airlines and tickets — the Flights table acts as the essential bridge.**

SQL Query

```
SELECT a.AirlineID,
    a.Name,
    COUNT(t.TicketID) AS [total tickets sold]
FROM Airlines a
JOIN Flights f  ON a.AirlineID = f.AirlineID
JOIN Tickets t  ON f.FlightID  = t.FlightID
GROUP BY a.Name, a.AirlineID
ORDER BY [total tickets sold] DESC;
```

Query 5• Top Airline by Departures at Each Airport

Business Problem	Strategic management wants to know which airline dominates departures at each airport — critical intelligence for slot allocation negotiations and competitive positioning.
Business Value	Reveals which carrier owns each hub in terms of flight volume. Used in slot auction strategy, lounge placement decisions, and identifying where to target new airline partnerships.
Key Insight	**RANK() OVER (PARTITION BY airport name) restarts ranking independently for each airport. Outer WHERE rnk = 1 cleanly picks the winner at every hub — handles ties automatically.**

SQL Query

```
SELECT Origin_Airport, Airline, total_flights
FROM (
    SELECT ao.Name  AS Origin_Airport,
           a.Name   AS Airline,
           COUNT(*) AS total_flights,
           RANK() OVER (
               PARTITION BY ao.Name
```

```
                ORDER BY COUNT(*) DESC
            ) AS rnk
    FROM Flights f
    JOIN Airlines a  ON f.AirlineID = a.AirlineID
    JOIN Airports ao ON f.Origin    = ao.AirportID
    GROUP BY ao.Name, a.Name
) t
WHERE rnk = 1;
```

Query 6 • Highest Ticket Price per Route (Origin → Destination)

Business Problem	The pricing team needs the maximum ticket price achieved on each origin-destination route to benchmark fares, detect pricing outliers, and set data-driven fare caps.
Business Value	Routes with very high max prices may indicate unmet demand ripe for capacity expansion. Routes with low maximums may be under-monetised — prime candidates for dynamic pricing uplift.
Key Insight	**CTE first aggregates max ticket price per flight. A correlated subquery then compares each flight's max to the route-level maximum — a two-layer filter that precisely isolates the top-priced flight per route.**

SQL Query

```
WITH route_price AS (
    SELECT f.FlightID,
           f.Origin,
           f.Destination,
           MAX(t.Price) AS highest_ticket_price
    FROM Flights f
    JOIN Tickets t ON f.FlightID = t.FlightID
    GROUP BY f.FlightID, f.Origin, f.Destination
)
SELECT *
FROM route_price rp
WHERE highest_ticket_price = (
    SELECT MAX(highest_ticket_price)
    FROM route_price
    WHERE Origin      = rp.Origin
      AND Destination = rp.Destination
);
```

Section 3 — Passenger CRM & Loyalty

Passenger-level intelligence helps the CRM and loyalty teams understand individual travel histories and identify the highest-value customers within each loyalty tier — increasing retention and lifetime value.

Query 7 • Passenger First Flight, Last Flight & Total Flights

Business Problem	The loyalty team needs each passenger's full travel history summary — first flight date, most recent flight date, and total number of flights — to drive targeted engagement.
Business Value	Identifies frequent flyers for tier upgrades, detects dormant passengers for win-back campaigns, and feeds churn risk models. One query replaces an entire CRM data export.
Key Insight	**MIN(DepartureTime) and MAX(DepartureTime) alongside COUNT in a single GROUP BY elegantly produce the full per-passenger travel timeline — no subqueries or self-joins needed.**

SQL Query

```
SELECT p.PassengerID,
    p.Name,
    MIN(f.DepartureTime) AS first_flight_date,
    MAX(f.DepartureTime) AS last_flight_date,
    COUNT(f.FlightID)    AS total_flights
FROM Passengers p
JOIN Tickets t ON p.PassengerID = t.PassengerID
JOIN Flights f  ON t.FlightID   = f.FlightID
GROUP BY p.PassengerID, p.Name;
```

Query 8 • Top Spender in Each Frequent Flyer Status Tier

Business Problem	The CRM team wants the single highest-spending passenger within each loyalty tier (None, Silver, Gold, Platinum) — for personalised VIP outreach and premium upgrade offers.
Business Value	The top spender in each tier is the most likely upgrade candidate and the most receptive to premium credit card or lounge access offers. Precision targeting saves marketing budget.
Key Insight	**SUM aggregates total spend per passenger after GROUP BY. RANK() OVER PARTITION BY FrequentFlyerStatus then ranks within each tier independently — outer WHERE rnk=1 extracts the champion of each group.**

SQL Query

```
SELECT *
FROM (
    SELECT p.PassengerID,
           p.Name,
           p.FrequentFlyerStatus,
           SUM(t.Price) AS Total_Spending,
           RANK() OVER (
               PARTITION BY p.FrequentFlyerStatus
               ORDER BY SUM(t.Price) DESC
           ) AS rnk
    FROM Passengers p
    JOIN Tickets t ON p.PassengerID = t.PassengerID
    GROUP BY p.PassengerID, p.Name, p.FrequentFlyerStatus
) x
WHERE rnk = 1;
```

SQL Techniques Reference

All key SQL patterns used across the 8 queries — with purpose and the queries where each appears.

SQL Technique	Purpose	Queries
`JOIN + GROUP BY + FETCH NEXT`	Aggregate flight counts per airport; FETCH NEXT 1 ROW limits output to the single top result.	Q1
`3-table JOIN chain`	Connects Airlines → Flights → Tickets since no direct link exists. Flights acts as the bridge.	Q2, Q3
`Double alias self-join`	Airports table joined twice (ao = origin, ad = destination) to resolve both names in one query.	Q3
`RANK() OVER PARTITION BY`	Window rank restarts per airport group. Outer WHERE rnk=1 picks the dominant airline per hub.	Q4, Q8

SQL Technique	Purpose	Queries
`CTE + DATEDIFF`	CTE pre-computes flight hours cleanly; CASE WHEN then buckets into Short/Medium/Long labels.	Q5
`MIN / MAX aggregation`	MIN(DepartureTime) = first flight, MAX(DepartureTime) = last flight — elegant timeline query.	Q6
`CTE + Correlated Subquery`	CTE isolates per-flight max price; correlated subquery filters to route-level max ticket.	Q7
`SUM + RANK() OVER`	SUM aggregates total spend per passenger; RANK partitioned by loyalty tier finds the winner.	Q8

Database Schema

The Airport_data database contains 5 tables connected through foreign key relationships.

Table	Primary Key	Description
`Airports`	`AirportID (PK)`	Airport name, location, and country
`Airlines`	`AirlineID (PK)`	Airline name and home country
`Flights`	`FlightID (PK)`	Origin FK, Destination FK, departure/arrival times, AirlineID FK
`Passengers`	`PassengerID (PK)`	Passenger name, date of birth, frequent flyer status
`Tickets`	`TicketID (PK)`	PassengerID FK, FlightID FK, ticket price

Report Complete 8 queries • 3 sections • 5 tables